You can instruct Spring Security to provide CSRF tokens as cookies through the `csrf()` DSL exposed by `ServerHttpSecurity` and the `CookieServerCsrfTokenRepository` class. For imperative applications, that would be enough. However, for reactive applications like Edge Service, you need to take an extra step to ensure the `CsrfToken` value is actually provided.

In chapter 8, you learned that reactive streams need to be subscribed to in order to activate them. At the moment, `CookieServerCsrfTokenRepository` doesn't ensure a subscription to `CsrfToken`, so you must explicitly provide a workaround in a `WebFilter` bean. This problem should be solved in future versions of Spring Security (see issue 5766 on GitHub: https://mng.bz/XW89). For now, update the `SecurityConfig` class as follows.

Listing 11.16 Configuring CSRF to support a cookie-based strategy for SPAs

```
@EnableWebFluxSecurity
public class SecurityConfig {
  ...

  @Bean
  SecurityWebFilterChain springSecurityFilterChain(
    ServerHttpSecurity http,
    ReactiveClientRegistrationRepository clientRegistrationRepository
  ) {
    return http
      ...
      .csrf(csrf -> csrf.csrfTokenRepository(                    ❶
        CookieServerCsrfTokenRepository.withHttpOnlyFalse()))
      .build();
  }

  @Bean
  WebFilter csrfWebFilter() {                                    ❷
    return (exchange, chain) -> {
      exchange.getResponse().beforeCommit(() -> Mono.defer(() -> {
        Mono<CsrfToken> csrfToken =
          exchange.getAttribute(CsrfToken.class.getName());
```